

Tareas: Procesos y threads

Un *thread* (hilo) es la mínima secuencia de instrucciones que un SO puede planificar. Una tarea *multithreaded* puede crear varios hilos de ejecución concurrente.

Un **proceso** es una instancia de un programa en ejecución. Un *programa* es un **archivo ejecutable** que reside en algún dispositivo de almacenamiento.

Un *proceso* tiene al menos un hilo de ejecución. El código de un proceso se ejecuta en *modo usuario* (el menor modo de privilegio de la CPU) y puede acceder sólo a su *propio espacio de memoria*. El SO crea mecanismos de protección de acceso a la memoria. En caso que una instrucción intente acceder a una dirección fuera de su espacio, la CPU disparará una excepción. En el capítulo de [gestión de la memoria](#) se verán los mecanismos usados en detalle.

Es conveniente abstraer las diferencias técnicas entre procesos y threads y llamarlos *tareas (tasks)*.



Figura 1: Un proceso con múltiples threads.

Un *proceso multithreaded* puede crear hilos de ejecución adicionales, por ejemplo con la API [POSIX threads](#). Algunos lenguajes soportan threads nativamente, como *Java*, o *go*.

Cada thread usa su *propia pila* aunque todos los threads *comparten el espacio de memoria del proceso*.

El estado de un *thread* en un punto dado de su ejecución está representado por los valores de los registros de la CPU. En particular, los siguientes registros tienen una importancia particular.

- El *program counter* (*pc*): Dirección de la próxima instrucción a ejecutar.
- El *stack pointer* (*sp*): Dirección del *tope* de la pila.
- El *frame (o base) pointer* (*fp*): Dirección base del registro de activación correspondiente a la función o subrutina corriente.

La pila contiene *registros de activación* para el control de la secuencia de invocaciones a funciones y sus retornos. Además comúnmente contienen los *datos automáticos* (variables locales y argumentos) de las funciones activas. De ese modo se crean y eliminan automáticamente los *ambientes locales*.

La representación de un *registro de activación* se basa en el *calling convention* de la *ABI* de la plataforma.

El *frame pointer* se usa en las instrucciones del programa como dirección base para acceder en forma relativa (desplazamientos constantes) a las variables locales y argumentos, dentro del registro de activación.

Los pasos en una invocación a una función *f* son los siguientes:

1. Invocante (*caller*): Reserva espacio para el valor de retorno, si es que se dejará en la pila. Si se retorna en un registro, este paso se omite.
2. Invocante: Apila (o pone en registros) los valores de los argumentos.

3. Invocante: Ejecuta la instrucción `call fa`. *Salva* el contenido del *program counter* (`pc`) en un registro designado o lo *apila* y luego le asigna al `pc` la dirección `fa`.

4. Invocado (*called*): Apila el valor el *frame pointer* y le asigna el valor del *stack pointer*.

En el caso que `call` haya salvado el `pc` en un registro, el compilador puede generar código para *apilarlo* (`push`).

5. Invocado: Reserva espacio para las *variables locales*, restando un valor al *stack pointer*. Estos pasos (4-6) se conocen como el *prólogo*.

6. Invocado: Ejecuta las instrucciones de su *cuerpo*.

7. Invocado: Comienza el retorno (*epílogo*).

1. Se eliminan las *variables locales* y *temporarios* (`sp=fp`).

2. Se recupera el *base pointer* salvado (`pop fp`).

3. Ejecuta la instrucción de *retorno* (`ret`).

En algunas arquitecturas, como X86, `ret` espera la dirección de retorno en el tope del stack y lo desapila. En otras, como RISC-V `ret` copia en el `pc` el valor de `ra`. En el caso de haberse apilado, el compilador habrá generado instrucciones de *desapilado* en `ra` previo al `ret`.

8. Invocante: Desapila los argumentos, si hubiera, y toma el valor retornado (en la pila o desde registros de la CPU).

Esta secuencia de operaciones deja un *activation record* con el siguiente formato:

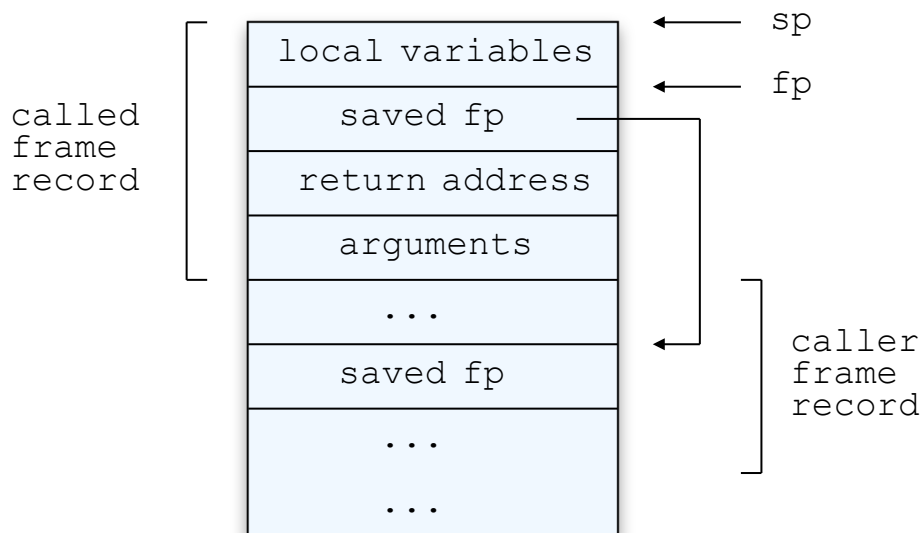


Figura 2: Pila de registros de activación de funciones.

El *calling convention* en una ABI de un SO o plataforma de hardware especifica los detalles de las invocaciones a funciones, en particular, cómo se pasan los argumentos (en registros, pila o ambos) y el valor retornado (en registros o en la pila).

Por ejemplo, Linux para x86-64 usa la especificación [System V ABI](#).

RISC-V: Una cpu *risc-v* tiene 32 registros de propósitos generales más el *program counter* (`pc`). Algunos de sus registros se usan según la siguiente convención:

- `x1` o `ra` : Return address (set by `call` or `jmp` instructions)
- `x2` o `sp` : Stack pointer.
- `x8` o `fp` : Frame pointer.
- `x10–x17` (`a0–a7`): Argumentos. En caso de funciones con más de 8 argumentos se deben apilar en el stack.
- El valor de retorno debe quedar en `a0` (y `a1` , opcionalmente). Los
- Los *saved registers* `s0–s11` deben ser salvados (en la pila) por la función invocada (si los usa).

Procesos de usuario

Un *proceso* representa un *programa de usuario en ejecución*. Generalmente se representa con al menos los siguientes atributos:

- Un identificador (*process id* o *pid*).
- Su estado: Ver la figura [estados de una tarea](#).
- Razón o recurso por el cual está en estado `SLEEPING` .
- Su *mapa* (áreas) de memoria a las cuales puede acceder en modo usuario.
 - Código (*text segment*).
 - Datos globales (*estático*): Área de datos inicializados (*data section*) y no inicializados (*bss section*).
- Al menos un *thread* de ejecución (su *main thread*), representado por
 - Stack en modo usuario. Stack que usa el *thread* mientras ejecuta en modo usuario.
 - Stack en modo kernel. En este stack se almacena el *trapframe*: Valores de los registros de la CPU al ocurrir una interrupción en éste thread. Esta pila también se usa para el control de invocación a funciones dentro del kernel ejecutándose en el *contexto del proceso*. Finalmente, también se usa para guardar el *contexto* (*caller saved CPU registers*) del thread cuando se produzca un *context switch*.
- Conjuntos de recursos adquiridos: Archivos/pipes/sockets abiertos, semáforos, timers, áreas de memoria compartida, manejadores de señales y otros.
- El *comando* o *programa* del cual es una instancia.

Los descriptores de las tareas se almacenan en una estructura de datos (arreglo, lista, árbol, etc).

Cuando las secciones de un programa se cargan en memoria desde el ejecutable, mediante el syscall `exec(path, args)`, ésta le asigna espacio para el stack del thread principal y se inicializa su *contexto de ejecución*, comúnmente en un campo del descriptor para que al retornar del kernel (retorno de la interrupción) la CPU pase a ejecutar la primera instrucción del programa (*entry point*). En particular se configuran los valores que deberán tomar son el *program counter (pc)* y el *stack pointer (sp)*.

El valor inicial del *sp* deberá apuntar a la primera instrucción de la función de inicio del proceso, comúnmente la función `_start` de la biblioteca estándar (enlazada con cada programa de usuario). Esta función comúnmente se define como:

C

```
void _start(void)
{
    exit(main());
}
```

El stack se inicializa de tal forma para que `main(int argc, char* argv[])` reciba los parámetros (argumentos de la línea de comandos) que correspondan.

La siguiente figura muestra el esquema (*layout*) en memoria de un proceso.

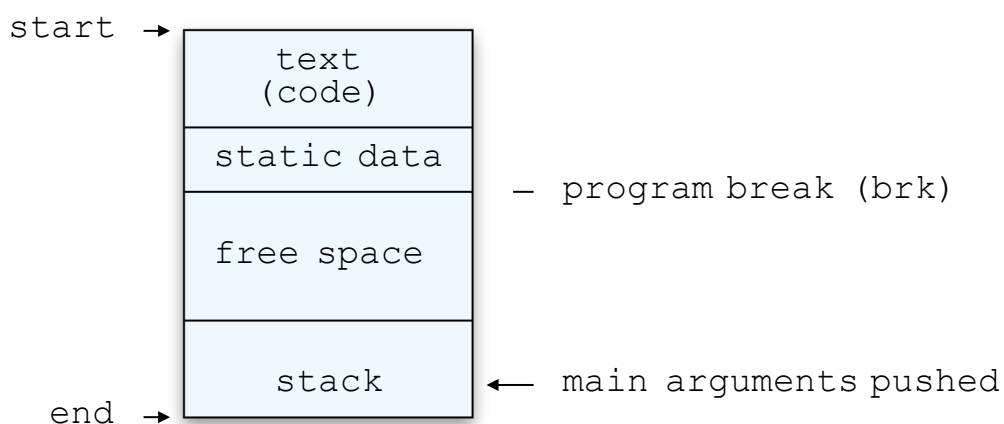


Figura 3: Layout (virtual) de memoria de un proceso.

El *espacio de direcciones* de un proceso es $(start, end)$. En muchos sistemas, $start = 0$.

En los SO tipo UNIX, la llamada al sistema `sbrk(n)` permite extender o reducir el área de datos globales del proceso asignando o liberando memoria y actualiza el *program break*. Puede que un proceso use un *heap* para almacenar variables dinámicas (creadas con `malloc()` y liberadas por `free()`). La memoria reservada para el *heap* puede estar por debajo del stack o a continuación de los datos estáticos.

El subsistema de gestión de procesos se encarga de la creación, cambios de estado, cambios de contexto, sincronización, comunicación y finalización de procesos. Mas abajo se describe en detalle el uso de los atributos mencionados. También se encarga de mantener el registro de los recursos adquiridos por cada proceso.

Un *proceso multithreaded* permite la creación de múltiples threads dentro del mismo espacio de memoria.

Modos de ejecución

Un *proceso de usuario* se ejecuta con la *CPU en modo usuario*. En éste modo la CPU no puede ejecutar ciertas instrucciones privilegiadas (como deshabilitar interrupciones, por ejemplo) y con acceso a sus propios espacios de memoria.

El código del kernel del SO se ejecuta en *modo kernel* o *supervisor*, un modo de mayor privilegio sin restricciones.

Traps: Interrupciones y excepciones

Los dispositivos (teclado, discos, etc) generan *interrupciones* para notificar eventos como la finalización de una operación.

La CPU puede generar *excepciones* al intentar ejecutar algunas instrucciones (como división por cero o accesos inválidos a memoria).

Una instrucción de *trap* generalmente se usa para implementar *llamadas al sistema* desde los procesos de usuario.

Los mecanismos que implementan el manejo de interrupciones, excepciones y traps son muy similares en la mayoría de las CPUs.

El SO debe instalar una o varias rutinas de manejo (*interrupt service routine (ISR)*) de estos eventos en su inicialización, configurando un **vector de interrupciones**. Cada dispositivo se configura para generar la interrupción *i* la cual será manejada por la *isr_i*. Las excepciones generadas por la CPU tienen números fijados por el fabricante.

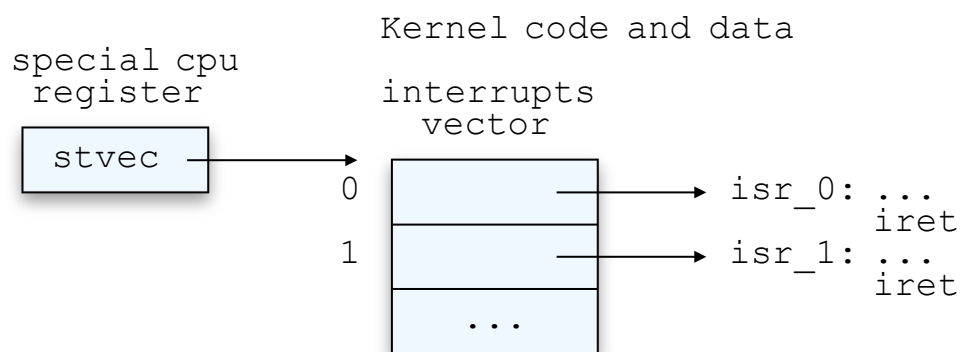


Figura 4: Vector de interrupciones.

La CPU ante uno de estos eventos realiza los siguientes pasos:

1. Salva el *pc* actual en algún registro de la CPU o en memoria.
2. Cambia el modo de ejecución a modo *supervisor*.
3. Salta a ejecutar la rutina correspondiente según el número de interrupción ocurrida (índice del vector).
4. Al retornar (instrucción *iret*), se restaura el modo de ejecución previo y se restaura el valor del *pc* con el valor salvado.

El kernel del SO tomará el control en cada interrupción/excepción/trap. Cada ISR invocará a las funciones del kernel correspondientes.

En algunas arquitecturas (como en *RISC-V*), es posible configurar la CPU para que todas las interrupciones disparen una única `ISR`. En éste caso la CPU deberá proveer algún mecanismo para determinar qué interrupción ocurrió. En el caso de *RISC-V*, el registro que apunta a la `ISR` es el `stvec` y en `scause` queda el código de la interrupción/excepción.

Llamadas al sistema de gestión de procesos

Algunas de las llamadas al sistema para la gestión de procesos se describen a continuación.

- `fork()` : Crea un nuevo proceso (hijo) el cual es una copia del proceso invocante. El proceso hijo *hereda* los descriptores de los archivos abiertos del padre.
- `exit(exitstatus)` : Finaliza el proceso.
- `wait(&childstatus)` : Espera (se bloquea) hasta que finalice un proceso hijo.
- `getpid()` : Retorna el identificador del proceso corriente.
- `kill(pid)` : Envía una señal (terminación) al proceso con id `pid`.
- `sleep(n)` : Espera (se bloquea) por `n` segundos.
- `exec(filename, args)` : Reemplaza la imagen de código y datos del proceso corriente por el código y datos de `filename`. Al final de esta sección se describe en mayor detalle.
- `sbrk(n)` : Asigna *n bytes* de memoria extra al proceso invocante.

Estados de un proceso

Un proceso puede estar en uno de los siguientes estados:

- **RUNNING**: Está ejecutando actualmente, tiene asignada una CPU.
- **RUNNABLE** o **READY**: Está esperando a que el kernel le asigne una CPU.
- **SLEEPING** o **WAITING**: Está *bloqueado* hasta la ocurrencia de un evento que lo despertará como la finalización de una operación de entrada/salida.

La [figura 3](#) muestra los cambios de estado que puede tener un proceso y las funciones de un kernel involucradas en las transiciones.

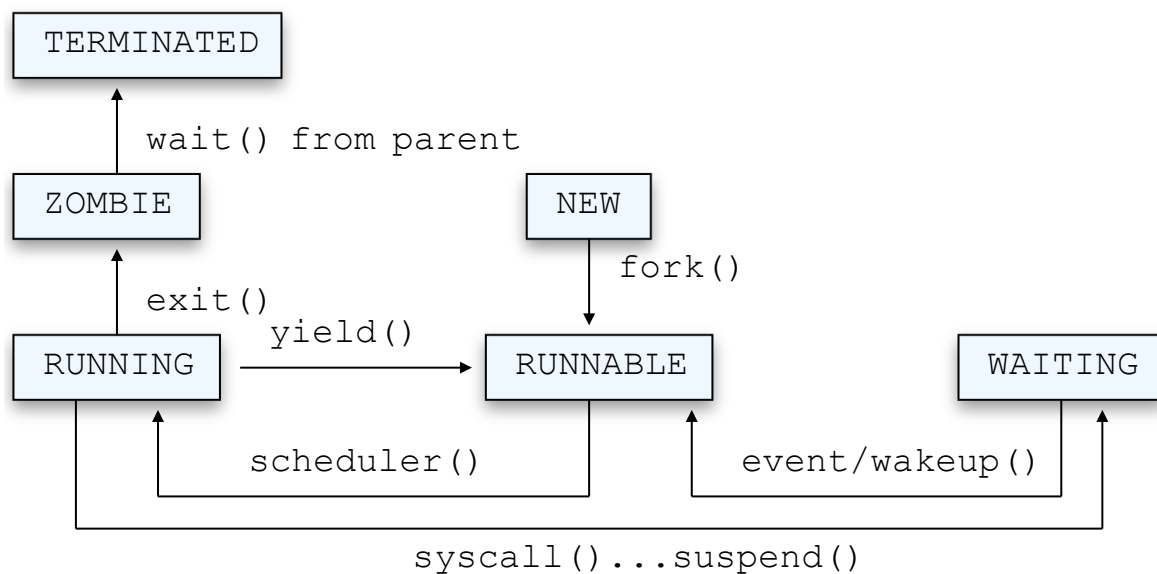


Figura 5: Cambios de estado de procesos.

En los sistemas tipo UNIX, un proceso que ha terminado su ejecución (vía `exit()`) pero aún existe en el sistema se denomina *zombie*. Esto ocurre comúnmente cuando el proceso *padre* aún no ha ejecutado `wait(&status)` y se requiere mantener su estado de terminación en el descriptor del proceso hasta que el proceso padre ejecute el `wait()` correspondiente.

Cada proceso, excepto `init` (el primer proceso lanzado por el kernel luego del *booting*) tiene un padre.

Nota: ¿Qué sucede si un proceso padre termina antes que sus hijos? En UNIX, los procesos hijos son *adoptados* por `init`.

Esto hace que el sistema mantenga un árbol de procesos (relación *padre/hijo*) donde `init` es la *raíz*.

Manejo de tareas

Generalmente habrá muchas tareas en un cierto estado por lo que generalmente se organizan en estructuras de datos. Por ejemplo, las tareas `RUNNABLE` se organizan en una cola o lista cuyo orden dependerá del algoritmo de *scheduling* utilizado.

Cuando un proceso está en estado `SLEEPING` es porque requiere esperar a que una operación finalice (por ejemplo, en una llamada al sistema como `read()`). En éstos casos estos procesos se encolan en una *wait queue* asociada al evento o dispositivo con información sobre el requerimiento realizado. Así, cuando el evento de finalización de la operación ocurra (ej: interrumpió el disco), el kernel puede determinar a qué tarea despertar (hacer `RUNNABLE`).

Otro aspecto a tener en cuenta es qué sucede si el *scheduler* no encuentra un proceso `RUNNABLE` (situación muy común). Se pueden implementar alguna de las siguientes alternativas:

1. Hacer que `scheduler()` quede en un ciclo hasta encontrar uno.
2. Usar un proceso `idle`, creado inicialmente por el sistema y que ejecute `while (true) yield();`. Este proceso al obtener la CPU la libera inmediatamente. Esto hace que siempre haya un proceso `RUNNABLE`.

En cualquier caso ésto es una oportunidad para *contabilizar* la carga de trabajo (o *idleness*) del sistema para realizar acciones como por ejemplo, ahorro de energía^[1] o disparar operaciones diferidas (como procesar paquetes de red recibidos) en un kernel thread.

Otra estructura de datos importante es la representación de la cpu, la cual comúnmente contiene comúnmente los siguientes atributos:

- Identificador de la CPU (por si hay más de una)
- Una referencia a la tarea corriente en ejecución.

- El contexto salvado del *scheduler* o *idle*: Estado de la CPU del último *cambio de contexto*.
- Estado de las interrupciones (habilitadas/deshabilitadas),
- Contadores (tiempo de ejecución, etc).

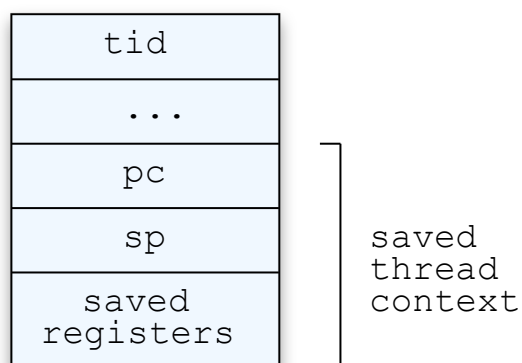
En un sistema que soporte *múltiples CPUs* (ej: multicore) deberá tener una estructura por CPU.

Cambios de contexto

Un SO representa una *tarea* en una estructura de datos que contiene su *estado* (`RUNNING` , `WAITING` , etc), su identificador, sus recursos (áreas de memoria, archivos abiertos, etc) y su *saved context*.

El *saved context* es una estructura de cada tarea (o thread) para mantener el estado desde que abandonó la CPU para poder continuar su ejecución, típicamente el `pc` (próxima instrucción a ejecutar), el `sp` y posiblemente algunos otros registros de la CPU.

task descriptor



Es común que un SO multithread tenga más tareas en ejecución concurrente que procesadores. Con una CPU, sólo una tarea puede estar en ejecución. Los demás estarán esperando por la CPU (`RUNNABLE`) o esperando por algún evento `WAITING` .

El *scheduler* del kernel de un SO selecciona la próxima tarea para asignarle la CPU para que continúe con su ejecución.

En el caso de *timesharing*, una *interrupción del reloj* puede hacer que el kernel decida *quitarle la CPU (preemption)* a la tarea actual porque ya usó su *time slice* o *quantum* y asignársela a otra tarea **RUNNABLE** .

El *quantum* es el período de tiempo máxima de uso de la CPU. Así las tareas van a usar la CPU por *ráfagas*, dando la ilusión al usuario que todas progresan como si cada una tuviese su propia CPU.

Cuando el SO decide *quitar* la CPU a una tarea y asignársela a otra, se debe realizar un *context switch*:

1. Salvar los valores de los registros de la CPU correspondientes en el *contexto* del descriptor de la tarea.
2. Cambiar al stack de la nueva tarea.
3. Restaurar los valores salvados en el *contexto* de la nueva tarea en los registros correspondientes de la CPU.

La implementación de este mecanismo en un kernel comúnmente define las siguientes funciones:

- `yield()` : El thread corriente *abandona la CPU*.
- `scheduler()` : Selecciona el próximo thread a darle la CPU.

Abajo se muestra un esquema de una posible implementación.

```
struct task* current; // current task descriptor
...
void yield()
{
    current->state = RUNNABLE;
    switch_context(&current->ctx, &scheduler_ctx); //
} //
//
void scheduler() { //
```

```

    ... //
    while (1) { //
        current = NULL; //
        ... //
        struct task* = select_runnable_task(); //
        if (task) { //
            context_switch(&scheduler->ctx, &(task->ctx)); //
        } //
        ... //
    }
}

```

Un thread que no invoque a `yield()` monopolizará la CPU. Para prevenir ésto, el kernel usa el *timer (reloj)* que genera interrupciones periódicamente. En una interrupción del timer, se dispara la *ISR* correspondiente y puede decidir *quitar* la CPU al thread corriente (invocando a `yield()`) ya que está en el contexto de la *task* corriente.

Esto se conoce como *preemptive multitasking* o *tiempo compartido*.

Llamadas al sistema

Comúnmente, a cada llamada al sistema se le asigna un identificador único. Es común asignarles números naturales (en xv6, ver `kernel/syscall.h`).

Las llamadas al sistema se implementan en dos partes:

- Invocación desde un proceso (*user mode*).

Generalmente se implementan como funciones en la biblioteca estándar que se enlace con cada programa de usuario. Cada función se implementa comúnmente poniendo el código de *syscall* en un registro designado de la cpu (`a7` en *risc-v*) y luego ejecutar una instrucción de *trap* o *software interrupt* (`int` en x86 o `ecall` en *risc-v*).

- Funciones de servicios en el kernel.

Al ejecutarse una instrucción de *trap* correspondiente a un *syscall*, se dispara el mecanismo de manejo de interrupciones y se ejecuta el *trap handler* (o *ISR*) correspondiente en el kernel el cual invocará a la función correspondiente en el kernel.

Generalmente las *syscalls* retornan algún valor o código de error a los procesos. En el caso de una API *POSIX*, los valores negativos (generalmente -1) son códigos de error.

Procesos zombie

Las dos llamadas al sistema *POSIX* siguientes están relacionadas:

- `exit(exitcode)` : Finaliza el proceso corriente con código de salida `exitcode` .
- `wait(&exitcode)` : El proceso corriente espera (se bloquea) hasta que finalice un hijo. El código de salida del hijo se *copia* en el parámetro de salida `exitcode` .

Esto ofrece un mecanismo de sincronización y comunicación básico entre procesos. El proceso padre puede saber cómo terminó el hijo.

Por convención se asume que el código o *status* de salida 0 es *terminación normal* (o exitosa). Un código de salida diferente a cero representa una finalización anormal y el código indica el motivo. Cada aplicación define el significado de los códigos de salida anormales.

Esto motiva el estado `ZOMBIE` de un proceso que ejecutó `exit()` antes que el padre ejecute `wait()` , ya que al menos se debe mantener el *código de salida* hasta que el padre ejecute `wait()` .

En el caso que el proceso padre finalice sin hacer un `wait()` , los sistemas tipo UNIX comúnmente hacen un *reparent* del proceso hijo, comúnmente haciendo que lo *adopte* `init` .

En un shell el código de salida del último proceso queda en la *variable de ambiente* `$?` .

El primer proceso de un sistema

En los sistemas tipo UNIX, el primer proceso lanzado es `init` .

El proceso `init` generalmente dispara el programa `getty` que se vincula a una *terminal*, luego éste dispara el proceso `login` .

El proceso `login` luego de la *autenticación del usuario*, ejecuta *shell* para que un usuario comience su interacción con el sistema. La siguiente figura muestra el árbol de procesos inicial en un sistema tipo UNIX.

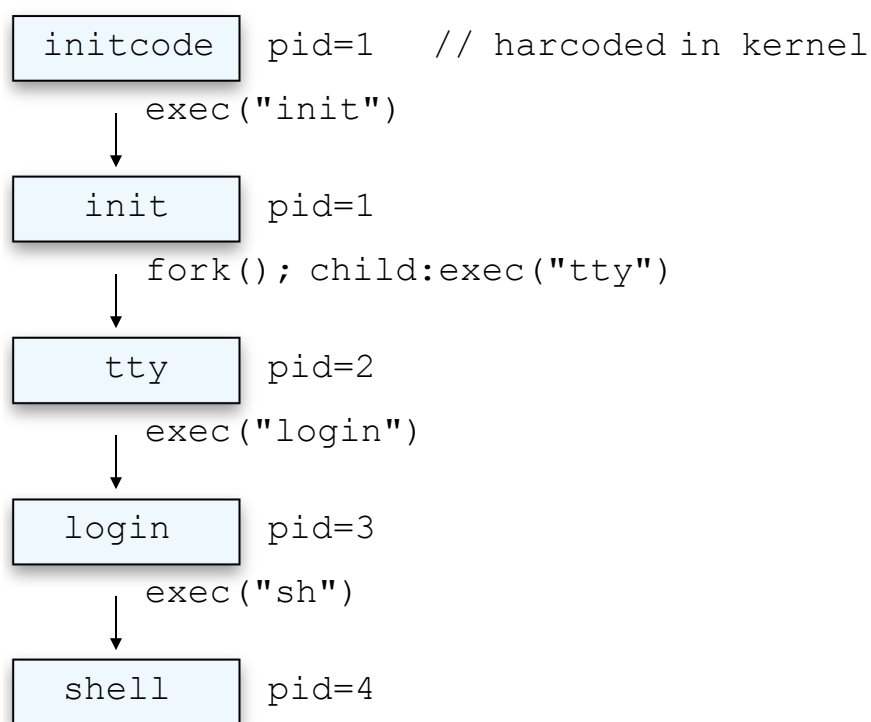


Figura 6: Inicio del sistema en sistemas UNIX.

En *EDOS* , haremos que `init` simplemente ejecute el shell ya que no soporta usuarios por ahora.

Implementación de tareas/procesos en el kernel

El siguiente listado muestra un esquema simplificado de implementación de un conjunto mínimo de operaciones en un kernel de un SO para el manejo de tareas.

c

```
// task.c module

// timer interrupts counter
uint ticks = 0;

//===== CPU dependent data structures =====
// saved context (resume point) of a task/thread not using
struct context {
    register_t pc;           // saved program counter
    register_t sp;           // saved stack pointer
};

// trapframe (saved cpu registers values) pushed on stack w
struct trapframe {
    register_t pc;           // saved program counter
    register_t sp;           // saved stack pointer
    // rest of cpu registers
    // ...
};

//=====

// task id counter
uint tid = 1;

// task descriptor
struct task {
    uint pid;
    uint state;
    struct trapframe *tf;    // pointer to saved cpu state st
    struct context ctx;      // saved context (for context sw
    // ...
};
```

```
// task descriptors array
struct task tasks[MAX_TASKS];

// cpu main thread saved context (running in scheduler func
struct context *sched_ctx;

// current task
struct task *current;

// trap (interrupts/exceptions) handler
// kernel entry point after the first task is launched and
trap(trap_number) {
    // interrupts disabled here
    // push cpu registers in current stack
    push_cpu_registers();
    switch (trap_number) {
        case TIMER:
            // timer interrupt
            ticks++;
            set_next_timer_interrupt();
            if (current_task_used_quantum())
                yield(); // preempt CPU
            break;
        case SYSCALL:
            // a process did a syscall
            syscall();
            break;
        case DISK_IRQ:
            // disk interrupt: read or write done
            wakeup(&disk_wait_queue);
            break;
        // other exceptions/interrupts...
    }
    // current can be changed here
    // restore cpu register values from stack
```

```

    restore_cpu_registers();
    // low level return from trap
    return_from_trap(); // in RISC-V: return from kernel mc
}

// create a new task. It will start at function fn
create_task(task_function fn) {
    struct task *new_task = get_task_free_slot();
    new_task->tid = tid++;
    new_task->ctx.pc = fn;
    new_task->ctx.sp = alloc_mem(STACK_SIZE) + STACK_SIZE;
}

// select next task to continue running and switch to it
// called from kernel_main() after boot
scheduler() {
    while (true) {
        enable_interrupts();
        current = select_runnable_task(); // scheduling algor
        if (current) {
            current->state = RUNNING;
            // save scheduler context and start/resume task ex
            context_switch(&sched_ctx, &current->ctx);
            // (1) we land here from yield() context switch
        }
    }
}

// current task leaves the cpu
yield() {
    if (current->state == RUNNING)
        current->state = RUNNABLE;
    // resume (continues) scheduler execution at line (1)
    context_switch(&current->ctx, &sched_ctx);
    // (2) task continue its execution
}

```

```
// kernel processing a syscall
syscall() {
    enable_interrupts();
    // syscall number and arguments are in the trapframe (or
    switch (syscall_number()) {
        // getpid() syscall
        case SYS_PID:
            // put result in saved trapframe (or below in stack)
            put_syscall_result(current->tid);
            break;
        // read(fd, buffer, count) syscall
        case SYS_READ:
            // Start disk read operation and send to sleep current
            do_read(current);
            suspend(&disk_wait_queue);
            break;
        // ...
    }
}
```

```
// suspend current task in wait queue and yield the cpu
suspend(struct wait_queue *wq) {
    current->state = WAITING;
    enqueue(wq, current);
    yield();
}
```

```
// wakeup a process waiting in wait queue
wakeup(struct wait_queue *wq) {
    struct task *task = dequeue(wq);
    if (task)
        task->state = RUNNABLE;
}
```

```
// low level context switch. Commonly written in assembly
```

```
context_switch(struct context *current, struct context *new)
{
    // copy pc and sp register values in current context
    save_ctx(current);    // assembly code
    // set cpu registers pc and sp with values from new context
    restore_ctx(new);    // assembly code
}
```

Figura 7: Esquema simplificado de manejo de procesos.

En éste listado se omite el código de inicialización del kernel luego del inicio (boot) y algunas operaciones de bajo nivel.

Ejercicio: Agregar los casos en `syscall()` para las llamadas al sistema `exit(exit_code)` y `child_pid = wait(&exit_status)` . Posiblemente tenga que agregar campos en `struct task` .

[1] El ahorro de energía puede apagar o suspender subsistemas y bajar la frecuencia de operación de la cpu.

< Anterior

Herramientas de desarrollo

Próximo >

Concurrencia e IPC

Concurrencia e IPC

En un kernel de un SO hay que considerar los problemas generados por la concurrencia.

Concurrencia: Mecanismos en un sistema que posibilitan la ejecución de múltiples hilos de ejecución por medio de ejecución simultánea (paralelismo) o *tiempo compartido (time-sharing)* con cambios de contexto. Estos mecanismos pueden generar diferentes secuencias de operaciones de manera no determinística o no controlada por el programador ya que puede depender del momento en que ocurren eventos y las decisiones del *scheduler*.

Aún con un único hilo de ejecución secuencial, el mecanismo de interrupciones o señales genera diferentes secuencias de ejecución (*intercalados* o *interleaving*) como se muestra en el siguiente ejemplo.

```
int v = 1;

// launched on some signal event
signal_handler() {
    v += 1;           // (1)
}

// main thread
// ...
v *= 2;              // (2)
print(v);            // (3) (output?)
```

Listado 1: Concurrencia generada por interrupciones o señales.

Algunas secuencias de ejecución (*interleaving*) pueden ser:

1. [(2), (3)] . Output 2 .
2. [(2), (1), (3)] . Output: 3 .
3. [(1), (2), (3)] . Output: 4 .

Dos o más *tareas o procesos cooperativos* usan algún recurso compartido.

En un proceso con múltiples *threads* es común el uso de variables compartidas (*shared memory*) ya que se ejecutan en el mismo espacio de memoria.

En el caso de procesos (cada uno tiene su propio espacios de memoria) será necesario algún mecanismo de comunicación entre procesos como *pasaje de mensajes* o *shared memory*.

Para comunicar *procesos remotos* es necesario usar *protocolos de comunicación* (*transmisión de mensajes*).

Cuando se usan *recursos compartidos* entre tareas (como memoria compartida), pueden ocurrir *condiciones de carrera*.

Los mecanismos de sincronización y comunicación ofrecidos por un SO se conoce como *inter-process communication (IPC)*.

Una **Condición de carrera (race condition)** surge cuando el comportamiento del sistema depende de la secuencia (*timing* o *interleaving*) de la ocurrencia de eventos externos que pueden generar flujos de ejecución no deseados.

La porción de código que accede a un recurso compartido se conoce como **región crítica**.

La concurrencia puede generar diferentes *trazas de ejecución* que no están bajo el control del programador ya que depende del *scheduler* (*no determinismo*).

En un sistema correcto, este *no determinismo* no debería alterar sus objetivos de computación.

La existencia de una *condición de carrera* en un sistema puede hacer que éste *no determinismo* sea *observable* (diferentes resultados en diferentes ejecuciones).

Surge la necesidad de impedir que se ejecuten *trazas* no deseadas. Esto requiere **mecanismos de sincronización**.

Locks

Un *lock* se asocia a un recurso para sincronizar su acceso de manera *exclusiva*.

Provee *exclusión mutua*, asegurando que cuando un thread adquirió el lock los demás que lo requieran deberán esperar a que el primero lo libere. Es común el uso del término *mutex* como sinónimo de lock.

Comúnmente un *lock* define operaciones `acquire(lock)` y `release(lock)`.

```
resource r;  
lock l = create_lock("queue lock");  
// ...  
acquire(&l);           // wait for resource is free  
use_resource(r);       // use resource inside critical region  
release(&l);           // release resource
```

Figura 1: Ejemplo de uso de *locks*.

Antes de realizar una operación sobre el recurso un thread deberá *adquirir* el *lock*, lo cual puede generar una *espera* hasta que el recurso sea liberado.

El uso de locks tiene consecuencias adversas como perder rendimiento porque las operaciones se secuencializan y produce *contención* en los threads que compiten por el recurso.

Los *locks* producen *contención* (esperas) y que hay que tratar de minimizarlas.

Spinlocks

Un *spinlock* hace que cada thread espere *usando* la cpu (en un ciclo). Esto se conoce como *espera ocupada*. Un primer intento de implementación podría ser el siguiente.

```
void acquire(int *lk)
{
    while (*lk)
        ;           // (1)
    *lk = 1;        // (2)
}

void release(int *lk)
{
    *lk = 0;
}
```

Listado 2: Primer intento de implementación de spinlocks.

Abajo hay un escenario con dos threads usando un lock `l` ;

```
// main program (sequential)
int l = 0;
// ...
// fork two threads
...
// thread 1                                thread 2
...                                         ...
acquire(&l);                               acquire(&l);
// critical region                          // critical region
release(&l);                               release(&l);
...                                         ...
```

Suponiendo que la cpu está ejecutando el thread 1 dentro de `acquire()` en la línea (1) y `l == 0`. Antes de la línea (2) ocurre una interrupción de reloj y el *scheduler* del kernel decide hacer un *context switch* al thread 2.

Ahora el thread 2 también invoca a `acquire()`. El valor de `l` aún es cero, por lo que ejecutará la línea (2) y continuará su ejecución, entrando a la región crítica.

En este punto, ocurre otra interrupción y el kernel decide hacer un *context switch* al thread 1, el cual continúa en la línea (2), retorna y también entra en la región crítica. No se ha podido garantizar la *exclusión mutua*.

La traza descrita corresponde a la siguiente secuencia:

`[t1-(1), t2-(1), t2-(2), t2-cr, t1-(2), t1-cr]`

donde `ti-(x)` corresponde a *thread i*, en *acquire-línea (x)* y `ti-cr` *thread i* en *critical region*.

El problema es que el lock en sí mismo es un recurso compartido y las operaciones en `acquire()` y `release()` deberían ejecutarse *atómicamente* (sin interrupciones en el medio).

La atomicidad se puede lograr deshabilitando las interrupciones en la cpu. El kernel no podrá tomar el control y hacer *context switches* hasta que se habiliten las interrupciones.

c

```
void acquire(int *lk)
{
    cli(); // clear (disable/ignore) interrupts
    while (*lk)
        ; // (1)
    *lk = 1; // (2)
}

void release(int *lk)
{

```

```

*lk = 0;
sti(); // enable interrupts
}

```

Listado 3: Segundo intento de implementación de spinlocks.

Esta solución funciona en monoprocesadores. En sistemas *multiprocesadores* no funciona ya que la operación `cli()` afecta sólo al procesador que la ejecuta.

Solución de Peterson

Peterson propuso en 1981 un algoritmo enteramente por software para resolver el problema de la *región crítica* o *exclusión mutua* basado en un modelo de memoria compartida. La propuesta inicial sólo contempla el problema de la exclusión mutua entre dos threads como se puede ver en el siguiente listado.

```

bool flag[2] = {false, false};
int turn;

lock(i)                                unlock(i)
{
    flag[i] = true;                    {
    other = i==0? 1: 0;                {
    turn = other;                      flag[i] = 0;
    while (flag[other] && turn == other)
        ;
}
}

// thread 0                            thread 1
lock(0);                                lock(1);
// critical section                    // critical section
// ...                                  // ...
unlock(0);                              unlock(0);

```

Listado 4: Solución de Peterson.

Esta solución se basa en usar `flag[i]` para representar la *intención* del thread *i* de ingresar en la región crítica y la variable `turn` para indicar qué thread tendría la prioridad para ingresar.

Es posible extender el algoritmo a *N* threads, como lo hace el algoritmo *filter* mostrado abajo. Este algoritmo funciona sobre *N* threads identificados por valores $[0..N-1]$ y usa una tabla `levels[N]` donde `levels[i]` representa el *nivel* o *posición* en una cola de espera para ingresar a la región crítica. El nivel cero indica que está a la cabeza de la cola de espera.

C

```
int level[N], last_to_enter[N];
enter(i)
{
    for (int l = 0; l < N; l++) {
        level[i] = l;
        last_to_enter[l] = i;
        while (last_to_enter[l] == i && same_or_higher(i,l)
               ;
    }
}

same_or_higher(i,j)
{
    for (k=0; k<N; k++) {
        if (k != i && level[k] >= j)
            return true
    }
    return false;
}

leave(i)
{

```

```
    level[i] = -1;  
}
```

Listado 5: Algoritmo *filter*.

Problema: ¿Este algoritmo garantiza progreso a cada thread?

Estas soluciones comúnmente **no funcionan** en sistemas multiprocesadores por el uso de *memorias caché*, por lo que se requieren modificaciones.

Spinlocks en multiprocesadores

En un multi-procesador, las interrupciones se desactivan por cpu, por lo que dos cpus diferentes podrían ejecutar en paralelo `acquire(&l)` y podrían darse dos trazas paralelas las cuales ambas entrarían en la región crítica.

Se podría implementar un *big lock* deshabilitando las interrupciones en todo el sistema, ésto generalmente es mas costoso y produce mayor contención.

El principal problema es que en `acquire()` la condición de espera consiste en al menos dos operaciones que pueden ser interrumpidas en el medio.

1. *Test*: Se carga de la memoria y se analiza el valor del lock.
2. *Set*: Se escribe el valor 1 en el lock.

Es necesario lograr estos dos pasos en manera atómica.

Las cpus modernas *multicore* con *procesamiento simétrico (SMP)* proveen instrucciones atómicas para poder implementar locks. Hay de diferentes tipos:

- *Compare and swap (CAS)*: `cas(address, old, new)` .Chequea si una variable en memoria contiene un valor esperado (`old`), si es así la modifica con `new` .
- *Test and set (TAS)*: `old = tas(address, new)` .Asigna en `address` el valor `new` y retorna el valor anterior (`old`).

- *Fetch and add (FAA)*: `faa(address, value)` . Lee el valor desde memoria, se le suma `value` y se almacena nuevamente en memoria.

Estas instrucciones se implementan usando alguna de las siguientes técnicas:

1. Bloqueo del bus de memoria.
2. Bloqueo de la línea de caché que contiene el valor del lock (*coherencia de caché*).
3. Deshabilitar interrupciones. Comúnmente usado en monoprocesadores o microcontroladores.

El compilador `gcc` provee la función

`__sync_lock_test_and_set(&lock, value)` cuya implementación es dependiente de la plataforma (CPU).

En risc-v, gcc emite instrucciones de la forma

asm

```
li a5, 1;
amoswap.w a5, a5, (lock_address);
```

La instrucción `amoswap.w rd, rs2, (rs1)` atómicamente carga el valor de memoria en la dirección en `rs1` y lo almacena en `rd` , luego *intercambia* los valores de `rd` y `rs2` para finalmente almacenar el valor de `rd` en la dirección de memoria contenida en `rs1` .

Una posible implementación de spinlocks en multi-procesadores se muestra abajo.

C

```
void acquire(int *lk)
{
    cli(); // clear (disable) interrupts
    while (__sync_lock_test_and_set(lk, 1) != 0)
        ;
    __sync_synchronize(); // memory barrier (fence)
    *lk = 1;
}
```

```
void release(int *lk)
{
    __sync_lock_release(lk); // *lk = 0;
    __sync_synchronize();
    sti(); // enable interrupts
}
```

Listado 6: Implementación de spinlocks en multiprocesadores.

La función `__sync_synchronize()` expande a una instrucción del tipo *fence* (o *barrera*) para prevenir que el compilador y la CPU genere o ejecute instrucciones *fuera de orden* con respecto a la secuencia del programa.

Comúnmente esto se hace para mejorar el rendimiento o hacer un mejor aprovechamiento de los registros y de los *pipelines* de la CPU.

Las instrucciones que se consideran *independientes*, es decir, que no tienen dependencias entre sus operandos podrían ejecutarse en cualquier orden.

Además una *barrera* asegura que los datos en caché se escriben en memoria antes de pasarla e invalidan las cachés de las demás CPUs. Esto hace que otra CPU cuando tenga que leer la variable `lk` tenga que acceder a la RAM.

Muchos multiprocesadores modernos comúnmente se usan *modelos de memoria relajados*, en el cual una escritura en memoria por una cpu puede no ser visible en otra ya que puede quedar en su caché local (*L1*). El objetivo es reducir el número de accesos a RAM ya que los tiempos de acceso son mucho mayores que a los registros o a la memoria caché.

Si el cambio en el valor del lock no es visible por otra cpu, la implementación no funciona.

Ver la implementación de *spinlocks* dados en el paso *02-hello-smp* de *edos-steps*.

Sleep locks

Un spinlock sólo es útil para realizar esperas cortas, es decir, cuando la región crítica toma muy poco tiempo ya que se está usando la CPU en la espera.

En el caso que la espera dependa de una condición que puede cambiar dentro de demasiado tiempo es preferible *suspender* la ejecución del thread o proceso y reiniciarlo cuando la condición haya cambiado. Esto es típico cuando en una región crítica se deben hacer operaciones de entrada-salida o computaciones largas.

Un *sleep lock* realiza esta tarea. Son similares a las *variables de condición*, las cuales asocian un conjunto de procesos a una condición de espera (el lock). Una API de sleep locks podría ser la siguiente:

- `void sleep(task* t, wait_queue* q)` : Pasa `t` a estado `SLEEPING` dejándolo encolado en la cola `q`.
- `task* wakeup(wait_queue* q)` : Desencola una tarea de `q` pasándola a estado `RUNNABLE`.

Obviamente, la implementación de estas primitivas requiere el uso de primitivas de locks como los *spinlocks* ya que la cola es un recurso compartido y `sleep` y `wakeup` pueden ejecutarse concurrentemente.

Uso de locks

Uno de los principales problemas es cuando el uso de locks no está encapsulado en una única función, sino que una lo adquiere y se debe liberar en otra. Esto muestra que *los locks no son modulares*. El uso de locks crea efectos colaterales que *deben ser documentados*.

El uso de locks debe ser cuidadoso porque pueden generar problemas como *deadlock*, *livelocks* y *double locks*.

Si un thread intenta adquirir un recurso previamente *adquirido* por él mismo, se bloqueará por siempre (*self lock* o *double lock*) y los demás no podrán acceder al recurso.

Este problema puede resolverse implementando *locks recursivos*.

Otro problema es el uso de múltiples locks. Suponiendo que se usan locks como se muestra a continuación.

C

```
f() {
    ...
    acquire(&l1);
    ...
    acquire(&l2);
    ...
    release(&l2);
    ...
    release(&l1);
}

g() {
    ...
    acquire(&l2);
    ...
    acquire(&l1);
    ...
    release(&l1);
    ...
    release(&l2);
}
```

(1)

Listado 7: Uso de locks.

En este caso hay un potencial *deadlock*, ya que un thread ejecutando `f()` en el punto (1) y otro thread ejecutando `g()` en el mismo punto quedarán esperándose mutuamente.

Este escenario produce una *espera circular*. La forma de evitarlo es siempre manteniendo el mismo orden en el uso de los locks. En un sistema complejo esto puede ser muy difícil de lograr.

Condiciones para que exista posibilidad de deadlock:

1. **Exclusión mutua:** Los procesos han adquirido un recurso en forma exclusiva.
2. **Espera y retención:** Un proceso retiene un recurso mientras espera por otro.
3. **No quita (preemption):** Los procesos sólo liberan sus recursos voluntariamente.

4. *Espera circular*: Existe una dependencia circular de recursos. Sean los procesos $P_0, P_1, \dots, P_{n-1}, P_i$, para todo i tal que $0 \leq i < n$, espera por un recurso adquirido por $P_{(i+1) \bmod n}$.

Generalmente en un SO se debe aplicar una disciplina particular en el uso de locks. Por ejemplo, se debe definir el *orden de uso* de locks en todos los módulos del sistema. Esta estrategia *rompe* la condición 4.

Un **livelock** ocurre cuando dos o más threads quedan en ciclos sin progresar en alguna computación útil.

Semáforos

Un semáforo (propuesto por E. Dijkstra en 1963) comúnmente representa una cantidad de recursos disponibles. Es una primitiva de sincronización más general que los locks.

Un semáforo *n-ario* (o **counting semaphore*) permite que hasta n threads accedan al recurso compartido. Cualquier thread adicional que intente acceder al recurso deberá esperar hasta que algún otro lo libere.

Comúnmente un semáforo se implementa con un contador más un *sleeplock*. Una API de semáforos generalmente tiene las siguientes operaciones:

1. `sem_init(s, value)` : Inicializa el valor del semáforo `s`.
2. `sem_wait(s)` : Si el valor de `s` está en cero, suspende al invocante. Sino, decrementa su valor.
3. `sem_signal(s)` : Incrementa el valor de `s` y *despierta* a los threads o procesos bloqueados en `s`.

Es fácil de ver que si $n = 1$ sólo 1 thread podrá acceder al recurso garantizando exclusión mutua por lo que conoce como *mutex*.

El siguiente programa muestra el ejemplo clásico del problema del *productor-consumidor* con semáforos. El problema consiste en que hay uno o más productores que producen valores en un buffer acotado de capacidad N y uno o más consumidores que extraen valores del buffer y los procesan.

Los productores y consumidores se ejecutan concurrentemente.

C

```

buffer buf[N];
semaphore mutex = 1, full = N, empty = 0;

void producer()
{
    while (true) {
        value = gen_value();
        sem_wait(&full);
        add_to_buffer(value);
        sem_notify(&empty);
    }
}

void add_to_buffer(value)
{
    sem_wait(&mutex);
    // add value to buffer
    sem_signal(&mutex);
}

void consumer()
{
    while (true) {
        sem_wait(&empty);
        value = get_from_buffer();
        sem_signal(&full);
        process(value);
    }
}

item_type get_from_buffer()
{
    sem_wait(&mutex);
    // extract item
    sem_signal(&mutex);
}

```

Listado 8: Productor-consumidor con semáforos.

El semáforo `full` representa el número de valores en el buffer y el semáforo `empty` representa el número de slots libres en el buffer.

Las operaciones sobre el buffer usan un semáforo binario o *mutex* para asegurar sus invariantes de representación ante la concurrencia.

Variables de condición

Una *variable de condición* es una primitiva de sincronización que maneja una *cola de tareas esperando una condición (estado)*. Comúnmente se usa un *lock* asociado al recurso. Su interface son las operaciones:

- `wait(&cond, &lock)` : Requiere que el invocante haya adquirido el lock. Atómicamente *libera* el lock agrega el thread invocante a la cola y lo suspende.
- `signal(&cond)` : Invocada por un thread que cambió la condición. Puede despertar uno o todos los threads esperando.

Ejemplo de uso en el productor-consumidor:

```

condition cond; mutex lock; buffer buf;
producer() {
    ...
    acquire(&lock);
    while (buffer_is_full())
        wait(&cond, &lock);
    produce(&buf);
    signal(&cond);
    release(&lock);
    ...
}

consumer() {
    ...
    acquire(&lock);
    while (buffer_is_empty())
        wait(&cond, &lock);
    consume(&buf);
    signal(&cond);
    release(&lock);
    ...
}

```

Analizar las operaciones `wait()` y `notify()/notifyAll()` de Java y determinar a qué tipo de mecanismo o primitiva de sincronización corresponde.

Monitores

Como ya se analizó, el uso de locks y semáforos no es simple. El principal problema es que no son modulares. Hansen y Hoare en 1974 propusieron un mecanismo de alto nivel llamado *monitor*. Un *monitor* es una construcción sintáctica de un lenguaje de programación que *encapsula* datos y las operaciones sobre ellos al estilo orientado a objetos. Los procesos o threads sólo pueden manipular y acceder a los datos sólo por medio de las operaciones del monitor. Es posible ver un monitor como un *objeto*, *clase* o *módulo* que es *thread safe*, es decir que los invariantes de sus operaciones están protegidos en presencia de concurrencia.

Su implementación común es usar *variables de condición* con al menos un *mutex* para asegurar la *exclusión mutua* entre las operaciones concurrentes sobre los mismos datos.

El siguiente listado muestra el ejemplo del productor-consumidor usando un monitor en un lenguaje hipotético:

c

```
monitor PC {
    condition full, empty;
    data buffer[N];
    int count = 0;

    void put(data item) {
        if (count == N) wait(full);
        insert(buffer, item);
        if (++count == 1) signal(empty);
    }

    data get() {
        if (count == 0) wait(empty);
        data item = remove(buffer);
        if (--count == N-1) signal(full);
    }
};

void producer() {
```

```
        while (true) {
            data item = gen_item();
            PC.put(item);
        }
    }

    void consumer() {
        while (true) {
            data item = PC.get();
            process(item);
        }
    }
}
```

Listado 9: Productor-consumidor con un monitor.

El monitor garantiza que las operaciones `put` y `get` se ejecuten de manera exclusiva entre ellos.

Algunos lenguajes con soporte de concurrencia proveen monitores. El lenguaje Java permite definir un monitor mediante el uso de *métodos sincronizados*. Una clase que declara métodos con el modificador `synchronized` asegura la ejecución con *exclusión mutua* en diferentes threads sobre el mismo objeto.

El uso de monitores permiten escribir programas concurrentes de una forma mas modular. Esta idea se puede aplicar en lenguajes sin monitores, usando una disciplina de modularización equivalente, *encapsulando* y *ocultando* el uso de locks.

Mensajes

Los mecanismos analizados anteriormente se basan en el modelo de concurrencia basado en *shared memory*. En el caso que dos o más procesos (cada uno ejecutando en su propio espacio de memoria) o sean remotos (en red) quieran comunicarse deberán usar algún mecanismo de *pasaje de mensajes*.

El modelo *message-passing* se basa en al menos dos primitivas del tipo

- `send(destination, message)`
- `receive(source, &message)`

La operación `send` envía datos a un proceso identificado como `destination` mientras que `receive` recibe un mensaje del emisor `source` (que puede ser `any`).

Estos mecanismos permitirían comunicar procesos locales o remotos (entre procesos ejecutándose en diferentes computadoras conectadas en red).

La comunicación puede implementarse usando *buffers de mensajes* o *mailboxes* o sin ellos.

En el primer caso la comunicación se dice *asincrónica* ya que el emisor no tiene que bloquearse ya que el mensaje quedará en el *mailbox*. Un receptor sólo se bloqueará si el *mailbox* está vacío.

Si no se usan buffers, la comunicación será *sincrónica* ya que los procesos deberán *coincidir* (esperarse) en las operaciones `send/receive` . Esto se conoce como un *rendezvous*.

En un SO con diseño *micro-kernels* como [MINIX](#), los subsistemas (gestor de procesos, memoria, sistemas de archivos, etc) se implementan como *servers* que reciben requerimientos de los demás subsistemas por medio de un mecanismo de mensajes y tienen la siguiente forma:

c

```
int main(void)
{
    message m;
    init_subsystem();
    while (TRUE) {
        receive(ANY, &m);
        switch (m.type) {
            case REQUEST_1:
                ...
                send(other_subsystem, m2);
            ...
        }
    }
}
```

```

        ...
        break;
    ...
}
}
}

```

Futuros y promesas

Un *futuro* o *promesa* refiere a una abstracción que representa un valor que será computado por algún otro componente (thread) de ejecución concurrente. Se usan comúnmente para implementar *data-flow variables*: Variables que ante una lectura se espera a que se asigne su valor (por otro thread). Las operaciones de lectura actúan como un mecanismo de sincronización.

En este modelo es común que se dispongan de las siguientes operaciones:

- `future f = async g(args)` : Invocación asincrónica de `g()` (en un nuevo thread). El invocante continúa con la ejecución. Su implementación comúnmente ejecuta `g` en un nuevo thread. Retorna un *future*: una variable *data-flow*.
- `value = f.get()` : Esta operación obtiene el valor (encapsulado en `f`) posiblemente bloqueando al invocante hasta que el thread evaluando `g()` finalice.

Este mecanismo es *built-in* en algunos lenguajes de programación como en [Oz](#) en los cuales la operación `get()` es implícita. En otros lenguajes como en C++, se implementan en bibliotecas y el acceso a un *future* es explícito (como `f.get()`).

Concurrencia sin locks (lock free)

Una estructura de datos que puede usarse concurrentemente y que no usa operaciones de bloqueo se denominan *lock-free*. Obviamente su ventaja es que es posible mejorar el rendimiento de un sistema y seguridad: Si un thread *finaliza* con un lock adquirido, los demás threads intentando acceder no podrán progresar.

Una estructura de datos es *wait-free* si cada thread que la usa puede completar sus operaciones en un número acotado de pasos.

Así, un *spinlock* califica como *lock-free* pero no como *wait-free*. El diseño de estas estructuras de datos no es simple. Su implementación generalmente requiere de instrucciones atómicas como las vistas arriba.

C

```
/* insert at front */
push(list* head, T value)
{
    list_node * n = create_node(value);
    n->next = head->first;
    // atomic compare and swap:
    // if (head->first == n->next) {
    //     head->first = n;
    //     return true;
    // }
    // return false;
    while (!cas(&(head->first), n->next, n))
        ;
}
```

Listado 10: Ejemplo de push() lock-free.

La estrategia consiste en identificar los puntos de programa en los que puede haber una *race condition* y asegurarse que se logró la actualización preservando el invariante. Se basa en la idea que la probabilidad que ocurra una traza no deseada es muy baja. Es difícil implementar estructuras de datos con operaciones de actualización en varios pasos.

IPC en UNIX

Los SO tipo UNIX se caracterizan por ofrecer una buena cantidad de mecanismos de sincronización y comunicación entre procesos como los que se listan en la siguiente tabla.

Mecanismo	Llamadas al sistema
File	<i>open, read, write, close, ...</i>
Pipes	<i>pipe, read, write, close, ...</i>
Named pipes	<i>mkfifo, read, write, close, ...</i>
Signals	<i>kill, signal, alarm, ...</i>
Message queues	<i>msgget, msgrcv, msgctl, ...</i>
Semaphores	<i>semget, semctl, semop, ...</i>
Shared memory	<i>shmget, shmat, shmctl, ...</i>
Memory mapped files	<i>mmap, munmap, ...</i>
Sockets	<i>listen, connect, accept, send, recv, ...</i>

Tabla 1: Mecanismos de IPC en sistemas UNIX.

Todos estos mecanismos permiten la comunicación y sincronización entre procesos locales (en una misma computadora) a excepción de los *sockets* que permiten comunicar procesos en diferentes computadoras (nodos) de una red.

[< Anterior](#)

[Próximo >](#)

Procesos y threads

Planificación de uso de cpu